

Event Sourcing + CQRS + NATS

DECISION POSTER · NATS-TECH-LAB DICTIONARY POC · DARK REFERENCE

THE QUESTION EVERY CARD SERVES

The correct responsibility split between **JetStream** (event backbone), **NATS KV** (lookup / watch / cache), **Postgres** (transactional store) and **CQRS projections**.

01 MODELING

Event-source it, or plain CRUD?

Does anything need to **replay this to reconstruct state**? — not “does it change?” State machine or temporal query ⇒ event-source.

Ship/Container ES · Ports
CRUD

9.5/9.6 ✓

02 CONSISTENCY

Which system owns the truth?

Postgres event store (A) vs JetStream (B) — **mirror images**: outbox and raw-projection are the same bridge from opposite banks.

Pattern B — only B tests
JetStream as a store

assumed ✓

03 PROJECTION

What shape is the read model?

A KV projection · **B** KV cache + Postgres · **C** replay from the log, no stored model. KV = snapshot vs projection — don’t confuse.

All three, **side-by-side**

1/2/6 ✓

04 CONSISTENCY

Cross-aggregate invariants

One stream → local & strong (one replay). **Two streams** → distributed: read-model guard / hydrate both / reservation / saga.

One stream first → split +
guard

8 ✓ · 12 ☹

05 IDENTITY

Surrogate or natural key?

Surrogate **UUID** when the natural key is an external standard you may correct; keep the natural key otherwise.

Container UUID · Ship natural

8.3 ✓

06 SAFETY · WRITE

Write-side safety

Optimistic concurrency (**Expected-Last-Subject-Seq**) + publish dedup (**Nats-Msg-Id**). Needs the {id} subject token.

Makes the log **trustworthy as a store**

10 ○

07 SAFETY · READ

Projection hardening

Sequence-guarded CAS writes; explicit MaxAckPending & retention; poison-message policy. Idempotent · ordered · checkpointed · versioned.

Safe under redelivery **by design**

11 ○

PATTERN FAMILIES

- Modeling
- Consistency
- Projection
- Identity
- Safety
- Performance → p.2

Cross-cutting concerns & POC map

SNAPSHOTS · PITFALLS · NATS ROLE MAPPING · PHASE STATUS

APPLIES TO EVERY DECISION ON PAGE ②

Replay cost, source-of-truth caveats, and the traps that bite once JetStream **is** the truth rather than just a bus.

SNAPSHOTS — TAMING REPLAY COST

PERFORMANCE

Load snapshot
@
lastStreamSequence

▶

Replay only after
events past that seq

▶

Rebuild +
validate

▶

Append

```
{ "aggregateId":"ship.SH-001", "state":{"..."},  
  "lastStreamSequence":9281 } # or you can't know what to replay
```

- Where write-side snapshot KV / PG — not a read-model projection
- Trust authoritative because replayed to latest seq
- Phase 13 Shape C unusable past a few thousand events without it; a busy ship's `hydrate()` slows its own writes

NATS BUILDING BLOCKS — ROLE MAPPING

FEATURE	WRITE / COMMAND	READ / QUERY
Streams	durable event log (truth)	rebuild projections
Subjects	partition region/tenant/agg/id	subscribe to a slice
Expected seq	concurrency guard	—
Durable consumers	sagas / process managers	the projector
KV	write-side snapshot (if replayed)	read-model projection (lags)

```
{region}.events.{tenant}.{aggregate}.{id}.{event}  
emea.events.acme.ship.SH-001.arrived  
emea.events.acme.container.<uuid>.loaded # id = UUID
```

THINGS TO BE AWARE OF

PITFALLS

Retention discards → truth erodes

LimitsPolicy, unbounded-is-deliberate & documented (Pattern B's worst default)

Exactly-once is a myth

at-least-once + dedupe → idempotent projections

Per-subject ordering only

no global timeline → aggregate-scoped projections

Hard rule off a read model

validate against write-side state, never a lagging KV

Correcting bad history

corrective event — never delete+republish

POC MAP — WHERE EACH DECISION IS EXERCISED

✓ DONE · ② IN PROGRESS · ○ PLANNED

9.5/9.6 ES vs CRUD DONE	assumed Truth = JS (B) LOCKED	1/2/6 Shapes A/B/C DONE	8 One-stream inv. DONE	8.3 Surrogate UUID DONE	9 Subject taxonomy DONE	10 Write-side safety PLANNED	11 Projection hard. PLANNED	12 Stream split PLANNED	13 Load / degrade PLANNED	14 Accounts OPTIONAL
--	--	--	---	--	--	---	--	--	--	---------------------------------------